

系统开发工具基础

第三周实验报告

乔海轩—25090012035

2026 年 9 月 7 日

目录

1 总览	3
2 第 9 题从源码构建并在干净环境安装 Wheel	4
3 第 10 题让编程智能体进入可验证的修复循环	5
4 第 11 题把“无法处理”的协作材料改成可执行信息	7
5 第 12 题修复一个可复现的线性回归训练循环	9
6 课后作业	10
7 课后作业	11
7.1 q1: 公里转英里 CLI 包	11
7.2 q2: 检查 sdist 内容	11
7.3 q3: 用 pip wheel 构建并检查入口点	11
7.4 q4: 修复除零返回 None	12
7.5 q5: 让时长解析支持分钟	13
7.6 q6: 负数 count 的 Issue/提交/评审	14
7.7 q7: README 缺少 Windows 安装步骤的 PR 描述	14
7.8 q8: 返回类型标注错误的评审意见	15
7.9 q9: 拟合 $y = 2x + 5$	15
7.10 q10: 小网络拟合 $y = x^2$	16

7.11 课后作业 commit 记录	16
8 解题感悟	17

1 总览

题目编号	描述	完成情况
9	从源码构建并在干净环境安装 Wheel	已完成
10	让编程智能体进入可验证的修复循环	已完成
11	把“无法处理”的协作材料改成可执行信息	已完成
12	修复一个可复现的线性回归训练循环	已完成

Github 仓库地址: <https://github.com/Ptilotsis-o/system-dev-tools/>
Commit 记录如图 1 所示。

```
* 87956a4 feat(q12): finished
* 8a46b99 feat(q12): file prepared
* 1e4cd24 feat(q11): rewrite the doc
* 78fb1bf feat(q10): write ai_log.md
* d2f85d9 feat(q10): fixed cli.py
* cb83519 feat(q10): craete test file
* babffea chore: ignore python's build/
* 6dc3913 feat(q09): files prepared
```

图 1: Commit 记录截图

2 第 9 题从源码构建并在干净环境安装 Wheel

题目要求

- 创建 `src/greetlab/__init__.py`、给定的 `cli.py` 和 `pyproject.toml`，并把“学号”替换为自己的学号。
- 在已安装 `build` 工具的课程环境中运行 `python -m build`，生成 `wheel`。
- 新建一个干净虚拟环境，只从生成的 `wheel` 安装。
- 切换到 `q09` 之外运行 `sdt-greet --name < 学号 >`，确认输出正确。

操作步骤与命令

```
mkdir -p q09/src/greetlab
vim q09/src/greetlab/__init__.py
vim q09/src/greetlab/cli.py
vim q09/pyproject.toml
cd q09
python -m build
python -m venv env
source env/bin/activate
pip install dist/greetlab_25090012035-0.1.0-py3-none-any.whl
cd ..
sdt-greet --name 25090012035
```

结果验证

- 三个文件创建完成，`pyproject.toml` 中的包名为 `greetlab-25090012035`；
- `python -m build` 执行成功，`dist/` 下生成 `greetlab_25090012035-0.1.0-py3-none-any.whl`；
- 新建的虚拟环境只安装了该 `wheel`，未从源码路径安装；
- 在 `q09` 之外运行 `sdt-greet --name 25090012035`，输出 `Hello, 25090012035!`。

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3$ sdt-greet --name 25898012035
Hello, 25898012035!
```

图 2: 在 q09 之外运行 sdt-greet 输出截图

3 第 10 题让编程智能体进入可验证的修复循环

题目要求

- 复制 q09 为 q10, 添加一个失败测试: 当 name 只含空白字符时, main 应以 `SystemExit(2)` 结束。
- 先运行测试确认失败, 再向编程智能体说明目标、约束和测试命令, 要求它修改实现并运行测试。
- 人工检查 diff, 撤销无关修改, 并再次运行测试。
- 在 `ai_log.md` 中用不超过 5 行记录核心提示、智能体改动和人工验证。

操作步骤与命令

```
cp -r q09 q10
mkdir -p q10/tests
vim q10/tests/test_cli.py
cd q10
pytest tests/test_cli.py -v
# 向编程智能体说明目标、约束与测试命令, 由智能体修改 src/greetlab/cli.py
git diff
pytest tests/test_cli.py -v
vim ai_log.md
```

结果验证

- 新增的 `test_blank_name_exits` 初次运行失败 (`DID NOT RAISE SystemExit`);
- 智能体仅修改了 `src/greetlab/cli.py`, 加入 `import sys` 与空白校验, `git diff` 中无无关改动;

- 再次运行 `pytest` 通过，正常姓名输入的行为未受影响；
- `ai_log.md` 用 3 行记录核心提示、智能体改动与人工验证，未超过 5 行。

```
(env) qlao-haiquan@qlao-haiquan-Virtual-Platform:~/system-dev-tools/week3/q10$ python -m pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qlao-haiquan/system-dev-tools/week3/q09/env/bin/python
cachedir: .pytest_cache
rootdir: /home/qlao-haiquan/system-dev-tools/week3/q10
configfile: pyproject.toml
collected 1 item

tests/test_cli.py::test_blank_name_exits FAILED [100%]

===== FAILURES =====
test_blank_name_exits

monkeypatch = <pytest.monkeypatch.MonkeyPatch object at 0x75fddadd5b0>

    def test_blank_name_exits(monkeypatch):
        monkeypatch.setattr(sys, "argv", ["sd-greet", "--name", ""])
        with pytest.raises(SystemExit) as exc_info:
            #
E       Failed: DID NOT RAISE SystemExit

tests/test_cli.py:13: Failed
----- Captured stdout call -----
Hello, 

===== short test summary info =====
FAILED tests/test_cli.py::test_blank_name_exits - Failed: DID NOT RAISE SystemExit
===== 1 failed in 0.04s =====
```

图 3: 添加失败测试后初次运行 `pytest` 截图

```
PS D:\Storage\work\SDTreports\system-dev-tools\week3\q10> git diff
diff --git a/week3/q10/src/greetlab/cli.py b/week3/q10/src/greetlab/cli.py
index 99969f9..ea6081f 100644
--- a/week3/q10/src/greetlab/cli.py
+++ b/week3/q10/src/greetlab/cli.py
@@ -1,6 +1,13 @@
import argparse
+import sys
+
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
+
+    if not a.name.strip():
+        sys.exit(2)
+
+    print(f"Hello, {a.name}!")
\ No newline at end of file
PS D:\Storage\work\SDTreports\system-dev-tools\week3\q10>
```

图 4: 智能体改动后的 `git diff` 截图

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3/q10$ python -m pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qiao-haixuan/system-dev-tools/week3/q09/env/bin/python
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week3/q10
configfile: pyproject.toml
collected 1 item

tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== 1 passed in 0.01s =====
```

图 5: pytest 通过截图

4 第 11 题把“无法处理”的协作材料改成可执行信息

题目要求

- 在 communication.md 中重写 Issue，包含环境、复现命令、期望结果和实际结果；未知信息写“待确认”。
- 重写提交信息：标题使用祈使语气，正文说明问题和解决方案。
- 重写评审意见：指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit。
- 全文控制在 400 字以内，保持专业、简洁。

操作步骤与命令

```
vim communication.md
wc -m communication.md
```

结果验证

- Issue 部分包含环境（操作系统“待确认”、Python 版本“待确认”、包版本 greetlab_25090012035-0.1.0）、复现命令 `sdt-greet --name "`”、期望结果与实际结果；
- 提交信息标题为祈使语气 `fix(cli): reject blank names with non-zero exit code`，正文分别说明问题与解决方案；
- 评审意见指出具体行为（未校验 `-name`）、潜在风险（退出码掩盖调用错误）与建议动作，并标注 `Blocking`；

- 全文 400 字以内，无“尽快修复”“写得不好”等无法执行的信息。

5 第 12 题修复一个可复现的线性回归训练循环

题目要求

- 补全训练循环，正确调用 `zero_grad`、`backward` 和 `step`。
- 训练后调用 `model.eval()`，并在 `torch.no_grad()` 中计算最终损失。
- 打印最终损失、`weight` 和 `bias`；最终损失应小于 0.001。
- 不得直接把 `weight` 和 `bias` 赋值为 3 和 -1。

操作步骤与命令

```
vim train.py
python train.py
```

结果验证

- 训练循环中每个迭代依次调用 `opt.zero_grad()`、`loss.backward()`、`opt.step()`，顺序正确；
- 训练结束后调用 `model.eval()`，并在 `with torch.no_grad():` 中计算最终损失；
- 打印出的最终损失小于 0.001，`weight` 与 `bias` 分别收敛到接近 3 和 -1 的值；
- 代码中未对 `weight` 和 `bias` 直接赋值，两个参数完全由梯度下降更新得到。



```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3/q12$ python train.py
Final Loss: 0.000000
Weight: 3.0000, Bias: -1.0000
```

图 6: `train.py` 代码与运行输出截图

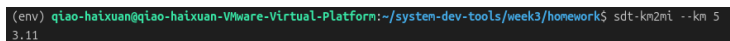
6 课后作业

7 课后作业

7.1 q1: 公里转英里 CLI 包

目标: 创建 `unitconv` 包, 提供命令 `sdt-km2mi`, 接受 `--km` 参数, 输出对应英里数并保留两位小数; 使用 `pyproject.toml` (setuptools 后端), 运行 `python -m build` 生成 wheel, 在干净虚拟环境中安装并运行验证。

```
python -m build
python -m venv env
source env/bin/activate
pip install dist/*
cd ..
sdt-km2mi --km 5
```



```
(env) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week3/homework$ sdt-km2mi --km 5
3.11
```

图 7: sdt-km2mi 运行截图截图

7.2 q2: 检查 sdist 内容

目标: 复用 q1 的包, 仅生成源码分发包 (sdist); 解压 `.tar.gz`, 将文件列表保存到 `sdist_files.txt`, 确认其中包含 `pyproject.toml`、`src/unitconv/cli.py` 和 `src/unitconv/__init__.py`。

```
python -m build --sdist
tar -tzf dist/*.tar.gz > sdist_files.txt
cat sdist_files.txt
```

7.3 q3: 用 pip wheel 构建并检查入口点

目标: 创建 `strtool` 包, 提供命令 `sdt-rev`, 功能为反转字符串; 使用 `pip wheel . -w dist` 构建 wheel, 再用 `unzip -l` 查看 wheel 内容, 并确认 `entry_points.txt` 中出现 `sdt-rev`。

```
pip wheel . -w dist
```

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3/homework/q32$ cat sdist_files.txt
unitconv_25090012035-0.1.0/
unitconv_25090012035-0.1.0/PKG-INFO
unitconv_25090012035-0.1.0/pyproject.toml
unitconv_25090012035-0.1.0/setup.cfg
unitconv_25090012035-0.1.0/unitconv/
unitconv_25090012035-0.1.0/unitconv/_init_.py
unitconv_25090012035-0.1.0/unitconv/cli.py
unitconv_25090012035-0.1.0/unitconv_25090012035.egg-info/
unitconv_25090012035-0.1.0/unitconv_25090012035.egg-info/PKG-INFO
unitconv_25090012035-0.1.0/unitconv_25090012035.egg-info/SOURCES.txt
unitconv_25090012035-0.1.0/unitconv_25090012035.egg-info/dependency_links.txt
unitconv_25090012035-0.1.0/unitconv_25090012035.egg-info/entry_points.txt
unitconv_25090012035-0.1.0/unitconv_25090012035.egg-info/top_level.txt
```

图 8: sdist_files.txt 内容截图

```
unzip -l dist/*.whl
unzip -p dist/*.whl '*/entry_points.txt'
```

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3/homework/q33$ unzip -l dist/*.whl
Archive: dist/strtool_25090012035-0.1.0-py3-none-any.whl
  Length      Date    Time    Name
-----
  0   2026-09-16 12:51   strtool/_init_.py
154  2026-09-16 12:51   strtool/cli.py
 63  2026-09-16 12:51   strtool_25090012035-0.1.0.dist-info/METADATA
 91  2026-09-16 12:51   strtool_25090012035-0.1.0.dist-info/WHEEL
 45  2026-09-16 12:51   strtool_25090012035-0.1.0.dist-info/entry_points.txt
  8  2026-09-16 12:51   strtool_25090012035-0.1.0.dist-info/top_level.txt
593  2026-09-16 12:51   strtool_25090012035-0.1.0.dist-info/RECORD
-----
 954                                7 files
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3/homework/q33$ unzip -p dist/*.whl '*/entry_points.txt'
[console_scripts]
sdt_rev = strtool.cli:main
```

图 9: entry_points.txt 内容截图

7.4 q4: 修复除零返回 None

目标: 修改 `divide.py`, 使 `divide(1, 0)` 返回 `None` 并向 `stderr` 输出 `division by zero`; 先编写失败测试, 再向编程智能体说明目标与测试命令, 人工检查 `diff`, 撤销无关改动, 最终运行 `pytest` 通过。

```
vim test_divide.py
pytest test_divide.py
# 与智能体协作, 修复 divide.py
pytest test_divide.py
git diff
vim ai_log.md
```

```
collected 1 item
test_divide.py::test_divide_zero FAILED [100%]

===== FAILURES =====
test_divide_zero

def test_divide_zero():
    assert divide(1, 0) is None
    """
test_divide.py:4:
-----
b = 1, b = 0

def divide(a, b):
    return a / b
    """
ZeroDivisionError: division by zero

Hlvide.py:2: ZeroDivisionError
===== short test summary info =====
FAILED test_divide.py::test_divide_zero - ZeroDivisionError: division by zero
===== 1 failed in 0.02s =====
```

图 10: pytest 运行报错截图

```
Python 3.8.5 Shell: 5.0.17 - /usr/bin/python3.8 test session starts
platform linux -- Python 3.8.5, pytest 5.4.1, pluggy 1.0.0 -- /usr/bin/python3.8
cachedir: .pytest_cache
rootdir: /home/runner/work/ai-log/ai-log, python: /usr/bin/python3.8
collected 1 item
test_divide.py::test_divide_zero PASSED [100%]

1 passed in 0.01s
```

图 11: pytest 运行通过截图

7.5 q5: 让时长解析支持分钟

目标: 修改 `parse_duration`, 使其支持"1h30m" 并返回总分钟数 90, 同时保持"2h" 返回 120; 编写失败测试, 让智能体修复后人工检查并运行测试。

```
vim test_parse_duration.py
pytest test_parse_duration.py
# 与智能体协作, 修复 parse_duration.py
pytest test_parse_duration.py
git diff
vim ai_log.md
```




图 14: pytest 运行通过截图

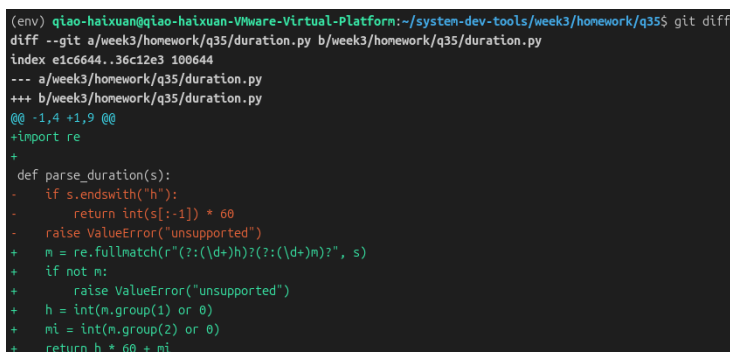


图 15: diff 内容截图

7.8 q8: 返回类型标注错误的评审意见

目标: 针对代码 `def add(a, b) -> int: return a + b`, 但调用 `add(1.5, 2)` 会返回 `float` 的问题, 撰写评审意见, 指出具体行为、风险和建议动作, 并标注 `Blocking`、`Suggestion` 或 `Nit`, 保存为 `review.md`, 不超过 200 字。

```
vim review.md
```

7.9 q9: 拟合 $y = 2x + 5$

目标: 用 PyTorch 拟合 $y = 2x + 5$; 使用 `nn.Linear(1, 1)`、`MSELoss`、`SGD(lr=0.05)`, 训练 300 步, 补全 `zero_grad()`、`backward()`、`step()`; 训练后调用 `model.eval()`, 在 `torch.no_grad()` 中打印最终 `loss`、`weight`、`bias`; 最终 `loss` 小于 0.001, 不得直接赋值 `weight/bias`。

```
vim train.py
python train.py
```

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3/homework/q39$ python train.py
Final Loss: 0.000000
Weight: 2.0000, Bias: 5.0000
```

图 16: 训练损失截图

7.10 q10: 小网络拟合 $y = x^2$

目标: 用 PyTorch 拟合 $y = x^2$; 使用 `nn.Sequential(nn.Linear(1, 8), nn.ReLU(), nn.Linear(8, 1))`, 优化器用 `Adam(lr=0.01)`, 训练 500 步; 补全训练循环, 并在 `eval() + no_grad()` 下打印最终 loss; 最终 loss 小于 0.05, 不得直接把网络权重写成解析解。

```
vim train.py
python train.py
```

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3/homework/q310$ python train.py
Final Loss: 0.00055
```

图 17: 训练损失截图

7.11 课后作业 commit 记录

```
* 8a0a824 (HEAD -> master, origin/master, origin/HEAD) feat(q310): train done
* bfdffda feat(q39): train done
* eb18711 feat(q38): write review.md
* b0019e8 feat(q38): file prepared
* cf3fcb8 feat(q37): write pr_description.md
* b5048be feat(q37): file prepared
* 9b6e3d8 feat(q36): write communication.md
* 377c45b feat(q36): file prepared
* 985eb87 feat(q35): write ai_log.md
* 65ecfe3 feat(q34): fix done
* 4d8b5d3 feat(q35): files prepared
* d31fe2e feat(q34): write ai_log.md
* 69946b4 feat(q34): fix done
* 060d145 feat(q34): files prepared
* 660457d feat(q33): files prepared
* a9ac66c feat(q32): create sdist_files.txt
* f292801 feat(q32): cp from q31
* 2d9663c feat(q31): files prepared
```

图 18: 课后作业 commit 记录

8 解题感悟

本次实验报告聚焦第三周内容，涵盖 Python 打包与 Wheel 安装、智能体编程与测试驱动修复、协作材料撰写以及 PyTorch 训练循环。通过动手实践，我掌握了从源码构建、干净环境验证到质量门禁的完整流程，也学会了向智能体清晰描述问题并人工审查改动。课后作业进一步巩固了 CLI 打包、除零修复、时长解析、回文判断、Issue/评审写作及线性/非线性回归。这次实验让我对打包发布、人机协作和深度学习训练有了更系统的认识。